Technical Assessment — Innovation R&D Engineer (Intermediate)

Candidate: Peter Kautwima
Position: Research & Development Software Engineer: Innovation (Intermediate)
Time Limit: Due by **12:00 (noon) Monday 14 July 2026**. No extensions.
Submission: GitHub/GitLab repository link + written responses (markdown or PDF)

Before You Begin

Ground Rules

1. **Use AI tools. Seriously.** You should use Claude, GitHub Copilot, ChatGPT, or any AI coding assistant to build this. That's how our team works — AI does the heavy lifting on code generation. If you're typing every line by hand, you're doing it wrong for this role.
2. **But you must be able to review and verify everything.** The job isn't writing code — it's directing AI to write code and then confirming it's correct, secure, performant, and does what you intended. We will ask you to walk through your submission and explain it. If you can't explain it, it doesn't count.
3. **Google, Stack Overflow, documentation — all fair game.** This is not a closed-book exam.
4. **Quality over completeness.** If you run short on time, a well-structured partial solution with clear documentation of what's missing is better than a rushed complete solution with poor code quality.
5. **Show your thinking.** Include a `DECISIONS.md` file in your repo explaining your architectural choices, trade-offs, and what you'd do differently with more time. This is weighted heavily in our evaluation.
6. **Commit often.** We'll look at your git history to understand how you approached the problem. Atomic commits with meaningful messages tell us a lot.

How We Work With AI

Our team operates as **AI operators with engineering foundations.** The workflow is:

1. **Direct** — tell AI what to build (this requires knowing what good looks like)
2. **Generate** — let AI produce the code (this is the fast part)
3. **Review** — read what it produced, catch issues, understand the logic (this is where engineering matters)
4. **Verify** — confirm it works, handles edge cases, integrates correctly (this requires real understanding)

You don't need to hand-write everything. But you DO need to understand every line well enough to catch when the AI gets it wrong — because it will. The code review section of this assessment tests that directly. The follow-up conversation tests it again.

The Challenge: Document Intelligence Service

You're building a small **Document Q&A service** — a system that lets users upload documents and then ask natural language questions about their content. This is a simplified version of the kind of AI-powered features our team builds daily.

The system has three parts: - A **Python backend** (API service) that handles document processing and question answering - A **React frontend** that provides the user interface - A **code review** where you demonstrate you can read and critique Python code

---

Part 1: Python Backend (Primary — ~45% of evaluation)

Build a FastAPI service that exposes the following capabilities:

Required Endpoints

Method	Path	Purpose
`POST`	`/documents`	Upload a text document (plain text or markdown). Store it, chunk it, and generate embeddings.
`GET`	`/documents`	List all uploaded documents (id, title, chunk count, upload date).
`GET`	`/documents/{id}`	Get document metadata and its chunks.
`DELETE`	`/documents/{id}`	Remove a document and its associated data.
`POST`	`/query`	Accept a natural language question. Find the most relevant chunks across all documents and return them with relevance scores.
`POST`	`/ask`	Accept a natural language question. Find relevant chunks, then use an LLM to generate an answer grounded in those chunks. Return both the answer and the source chunks used.

Technical Requirements

1. **Document chunking** — Split documents into meaningful chunks (you decide the strategy and chunk size — justify your choice in `DECISIONS.md`).
2. **Embeddings** — Generate vector embeddings for each chunk. Use an open-source embedding model such as:
   - `sentence-transformers` (e.g., `all-MiniLM-L6-v2`) — recommended, runs locally, free
   - Any HuggingFace embedding model
   - Another free/open alternative of your choice

   We don't care which specific model you pick — we care that you understand WHY embeddings matter and HOW similarity search works. Do NOT use paid APIs (OpenAI, etc.) for embeddings — use something open-source that runs locally.
3. **Vector storage & search** — Store embeddings and perform similarity search to find relevant chunks. Options:
   - In-memory (numpy cosine similarity) — perfectly acceptable
   - ChromaDB, FAISS, or any vector library — bonus points for reasoning about trade-offs
4. **LLM integration (for `/ask`)** — Call an LLM API to generate answers grounded in retrieved context. You may use:
   - OpenAI API
   - Anthropic (Claude) API
   - Any other LLM API
   - A mock that demonstrates you understand the prompt structure and context injection pattern

If you don't have API keys, mock the LLM call but write the actual prompt template you'd use. Explain your prompt design in `DECISIONS.md`.

5. **Request/response models** — Use Pydantic models for all request and response bodies. Proper validation and error responses.
6. **Error handling** — The service should handle failures gracefully (invalid input, missing documents, API failures). Return appropriate HTTP status codes and error messages.
7. **Project structure** — Organise the code as you would for a production service. We're looking at how you structure modules, separate concerns, and manage dependencies.

What We're Evaluating (Python)

Criteria	Weight	What "good" looks like
Service architecture	High	Clear separation of concerns (routes, services, models, storage). Not everything in one file.
Code quality	High	Readable, consistent style, meaningful names, appropriate abstractions.
Error handling	High	Graceful failures, proper HTTP status codes, informative error messages.
AI/RAG reasoning	High	Sensible chunking strategy, understanding of embeddings and similarity, good prompt design.
API design	Medium	RESTful conventions, clean request/response contracts, useful documentation.
Testing	High	Minimum 90% code coverage. Meaningful tests that cover happy paths, edge cases, and error scenarios. Use pytest. Include a coverage report in your submission.

Part 2: Code Review (Critical — ~20% of evaluation)

Below is a Python module from an existing service. It has **several issues** — some are bugs, some are architectural problems, some are style/maintainability concerns.

Your task: Review this code as if it came up in a pull request from a junior developer. Provide your review in a file called `CODE_REVIEW.md` in your submission.

For each issue you identify: 1. **What** the problem is (be specific — line reference or quote the code) 2. **Why** it's a problem (impact: bug? security? maintainability? performance?) 3. **How** you'd fix it (show the corrected code or describe the approach)

We expect you to find **at least 8 distinct issues**. There are more than 12.

```python
import os
import json
import requests
from fastapi import FastAPI, Request
from typing import Optional

app = FastAPI()

DATABASE_URL = "mysql://admin:password123@prod-db.internal:3306/documents"
API_KEY = "sk-proj-abc123def456"
```

```
documents = {}
counter = 0

@app.post("/upload")
async def upload_document(request: Request):
    body = await request.json()
    global counter
    counter = counter + 1

    doc = {
        "id": counter,
        "title": body["title"],
        "content": body["content"],
        "chunks": []
    }

    # chunk the document
    words = body["content"].split(" ")
    chunk_size = 100
    for i in range(0, len(words), chunk_size):
        chunk = " ".join(words[i:i+chunk_size])
        doc["chunks"].append(chunk)

    # get embeddings for each chunk
    for i in range(len(doc["chunks"])):
        response = requests.post(
            "https://api.openai.com/v1/embeddings",
            headers={"Authorization": f"Bearer {API_KEY}"},
            json={"input": doc["chunks"][i], "model": "text-embedding-ada-002"}
        )
        embedding = response.json()["data"][0]["embedding"]
        doc["chunks"][i] = {"text": doc["chunks"][i], "embedding": embedding}

    documents[counter] = doc
    return {"status": "ok", "id": counter}


@app.get("/search")
async def search(q: str, limit: Optional[int] = 5):
    # get query embedding
    response = requests.post(
        "https://api.openai.com/v1/embeddings",
        headers={"Authorization": f"Bearer {API_KEY}"},
        json={"input": q, "model": "text-embedding-ada-002"}
    )
    query_embedding = response.json()["data"][0]["embedding"]

    # find similar chunks
    results = []
    for doc_id, doc in documents.items():
        for chunk in doc["chunks"]:
            similarity = sum(a * b for a, b in zip(query_embedding, chunk["embedding"]))
            results.append({
```

```
                "doc_id": doc_id,
                "text": chunk["text"],
                "score": similarity
            })

    results.sort(key=lambda x: x["score"], reverse=True)
    return results[:limit]


@app.post("/ask")
async def ask_question(request: Request):
    body = await request.json()
    question = body["question"]

    search_results = await search(question)

    context = ""
    for r in search_results:
        context += r["text"] + "\n\n"

    prompt = f"Answer this question: {question}\n\nContext: {context}"

    response = requests.post(
        "https://api.openai.com/v1/chat/completions",
        headers={"Authorization": f"Bearer {API_KEY}"},
        json={
            "model": "gpt-4",
            "messages": [{"role": "user", "content": prompt}]
        }
    )

    answer = response.json()["choices"][0]["message"]["content"]
    return {"answer": answer}


@app.delete("/documents/{id}")
async def delete_document(id):
    del documents[id]
    return {"status": "deleted"}


@app.get("/documents")
async def list_documents():
    return documents
```

What We're Evaluating (Code Review)

Criteria	Weight	What "good" looks like
Issue identification	High	Catches security, correctness, and architectural issues — not just style nitpicks.
Explanation quality	High	Can articulate WHY something is wrong, not just that it "looks bad."

Criteria	Weight	What "good" looks like
Fix quality	Medium	Proposed fixes are correct and demonstrate understanding of the language.
Prioritisation	Medium	Distinguishes critical issues (security, bugs) from nice-to-haves (style).

Part 3: React Frontend (Secondary — ~15% of evaluation)

Build a simple React/TypeScript frontend that interacts with your backend service.

Required Features

1. **Document upload** — A form/interface to paste or upload text content with a title.
2. **Document list** — Show uploaded documents with metadata.
3. **Q&A interface** — A chat-like interface where the user can:
   - Type a question
   - See the AI-generated answer
   - See the source chunks that informed the answer (with relevance scores)
4. **Loading/error states** — Handle async operations gracefully.

What We're Evaluating (React)

Criteria	Weight	What "good" looks like
Component architecture	Medium	Logical component breakdown, separation of concerns, reusable where appropriate.
TypeScript usage	Medium	Proper typing (not `any` everywhere), interfaces for API responses, type-safe props.
State management	Low	Appropriate for the scale — hooks/context is fine. No need for Redux.
UX	Low	Functional and clear. We're not evaluating design skills — just basic usability.

Notes

- Use any React setup you prefer (Vite, Create React App, Next.js — your call).
- Styling is not important. Tailwind, plain CSS, unstyled — doesn't matter. Functional > pretty.
- This section should be fast for you given your experience. Don't over-engineer it.

Part 4: Architecture & Reasoning (Written — ~20% of evaluation)

Answer the following in your `DECISIONS.md` (or a separate `ARCHITECTURE.md`):

4.1 — Production Readiness

If this service needed to handle 1,000 documents and 100 concurrent users: - What would you change about your current implementation? - How would you handle embedding generation at scale (it's slow and expensive)? - What storage solution would you use for production and why?

4.2 — Deployment

Describe (briefly) how you'd deploy this service to AWS: - What AWS services would you use? - How would you handle the Python backend vs the React frontend? - What about the vector store / embedding model?

Note: We don't expect deep AWS expertise. We want to see how you think about deployment even if the specific services are new to you.

4.3 — Guardrails & Safety

The `/ask` endpoint generates responses using an LLM. In production: - How would you prevent the LLM from answering questions NOT grounded in the uploaded documents (hallucination)? - How would you handle potentially harmful or off-topic queries? - What would you monitor to ensure the system is working correctly?

4.4 — Your Approach

- How did you approach this assessment? What did you learn along the way?
- Where did AI tools help you most? Where did you have to rely on your own understanding?
- What was the hardest part, and how did you work through it?
- What would you build differently if you had 2 more days?

Bonus (Optional — shows initiative)

Pick ONE (or none — this is genuinely optional):

- **Conversation memory:** Make the `/ask` endpoint support follow-up questions that reference previous answers in the conversation.
- **Streaming responses:** Stream the LLM response to the frontend token-by-token instead of waiting for the full response.
- **Document format support:** Handle PDF upload (extract text) in addition to plain text.
- **Evaluation harness:** Create a simple test that measures answer quality (e.g., given known Q&A pairs from a document, does the system return correct answers?).

Submission Checklist

☐ Git repository with meaningful commit history
☐ `README.md` with setup instructions (we should be able to run it locally)
☐ `DECISIONS.md` covering Part 4 + architectural choices throughout
☐ `CODE_REVIEW.md` with your review of the provided code (Part 2)
☐ Working Python backend (even if LLM call is mocked)
☐ Working React frontend that communicates with the backend
☐ **Test suite with at least 90% code coverage** (pytest + coverage report included)
☐ `requirements.txt` or `pyproject.toml` for Python dependencies
☐ `package.json` for frontend dependencies

What We're NOT Testing

- Memorised knowledge of Python stdlib or obscure language features
- Deep NLP theory or ability to cite papers
- AWS certification-level infrastructure knowledge
- Design/CSS skills

- Speed (you have a week — take the time to do it well)

What We ARE Testing

- **Can you direct AI effectively** to build a structured, production-quality Python service in a domain that's new to you?
- **Can you review and verify** what AI produces — catching bugs, security issues, and architectural flaws in Python code?
- **Do you understand** how AI/LLM-powered features work architecturally (RAG, embeddings, context injection)?
- **Can you explain** every decision in your codebase — why it's there, what it does, what could go wrong?
- **Is the output** clean, well-reasoned, and maintainable — regardless of who (or what) wrote it?
- **Can you articulate** trade-offs, decisions, and growth areas honestly?

Follow-Up

We reserve the right to schedule a **30-minute technical walkthrough** if we have questions or need clarity on your submission. If scheduled, you'd be expected to: 1. Walk us through your architecture and key decisions 2. Explain specific code sections we select 3. Discuss what you'd change and how you'd extend the system 4. Answer questions about your code review findings

Your submission should stand on its own — but be prepared to defend it verbally if asked.

Good luck, Peter. We're excited to see what you build.